

Perception

Before

Desired angle/position
→ controller
→ actuator
→ physics

After

Camera frame
→ detect target
→ estimate target location
→ coordinate transformation
→ calculate desired robot state
→ controller
→ actuator
→ physics
→ new camera frame
↺

closed
perception - action
loop!

- The camera measures what happened.
- Vision converts pixels into useful target information.
- Geometry expresses that information in the robot's frame.
- IK converts a desired Cartesian position into desired joint angles.
- The controller compares desired and actual joint states.
- MuJoCo moves the robot.
- The camera observes the new situation.

> outer perception loop: → where the robot should go

camera → vision → mapping → IK → desired joints

> inner control loop:

desired joints → error → PD/PID → ctrl → actuators/robot → qpos
qvel

feedback

pan/tilt tracking: pixel error can directly command camera rotation

vision-guided arm: pixels must first become a world/robot target,
then IK produces desired joints

- Pan/tilt: image error can directly become desired camera rotation.
- Robot arm: image location must become a physical target before IK and joint control.

1. What does the outer perception loop output for the inner controller?
2. What does the inner controller output to MuJoCo?
3. Which loop directly reads `qpos` and `qvel`?

1. desired joints
2. ctrl
3. inner control loop

desired joints $\rightarrow$ controller $\leftarrow$ `qpos, qvel`
 $\downarrow$

controller $\rightarrow$ ctrl $\rightarrow$ robot moves

> complete conversion chain:

Image position (u, v) pixels	$\rightarrow$	Robot target (x, y) mm mapping	$\rightarrow$	Desired joints (θ_1^* , θ_2^*) rad IK	$\rightarrow$	Motor command ctrl controller
---------------------------------	---------------	--------------------------------------	---------------	---	---------------	-------------------------------------

then robot produces:
`qpos: rad`, `qvel: rad/s`



loops doesnot run at the same speed

Camera: ~ 30 frames per sec.

MuJoCo physics: timestep $0.002s$; 500 updates per sec

Controller: $\sim$ runs at physics rate

1. During those 16–17 control steps, does the desired joint angle change?
2. Do `qpos` and `qvel` still change on every physics step?
3. When a new camera frame arrives, which loop recalculates the desired joints?

1. no, only when outer loop sends new target
2. yes as the simulated robot moves
3. outer loop

→ holding the latest target unchanged until new info arrives!
zero-order hold

- Outer-loop error: where the target appears versus where we want it in the image/world.
- Inner-loop error: desired joint angle minus current `qpos`.

Target appears left of image centre

↓

Outer loop calculates new desired joints

↓

Inner loop compares desired joints with `qpos`

↓

PD/PID produces ctrl

↓

Robot moves

↓
Camera observes the new result

if camera lost the target, it has not produces valid pos. for IK

when detection disappears → reuse last known pos. → it's an old info. → this is called stale

↳ perception must output more than coords:

target position + detected / not detected + timestamp / confidence

1. If we pass every fluctuation directly into IK, what might the robot do?
2. Which loop should determine whether the target detection is valid?
3. If we keep using a target position detected five seconds ago, is that current or stale information?

1. first effect: jitter movement

2. outer loop; prev - IK

3. stale

⇒ for jitter-ing

- Filtering: smooth noisy measurements over time.
- Deadband: ignore tiny errors near the goal.
- Validity check: reject missing or unreasonable detections.
- Reachability check: reject targets that IK cannot reach.

> open-loop vs closed loop

detect
once
↓
observe
once

↓
detect
↓
move
↓
observe
result
↓
correct
↓
observe
again
↻

1. If someone pushes the robot away after an open-loop command finishes, will it automatically correct itself?
2. Would a continuously running camera feedback loop notice that disturbance?
3. Why might repeatedly observing the result be better than assuming the commanded movement happened perfectly?

1. no, there are several steps in advance

2. yes

3. repeated observations helps w/ external disturbances
 ↳ inaccurate models
 ↳ motor/physics imperfections

> ground truth: the exact target pos. to the controller

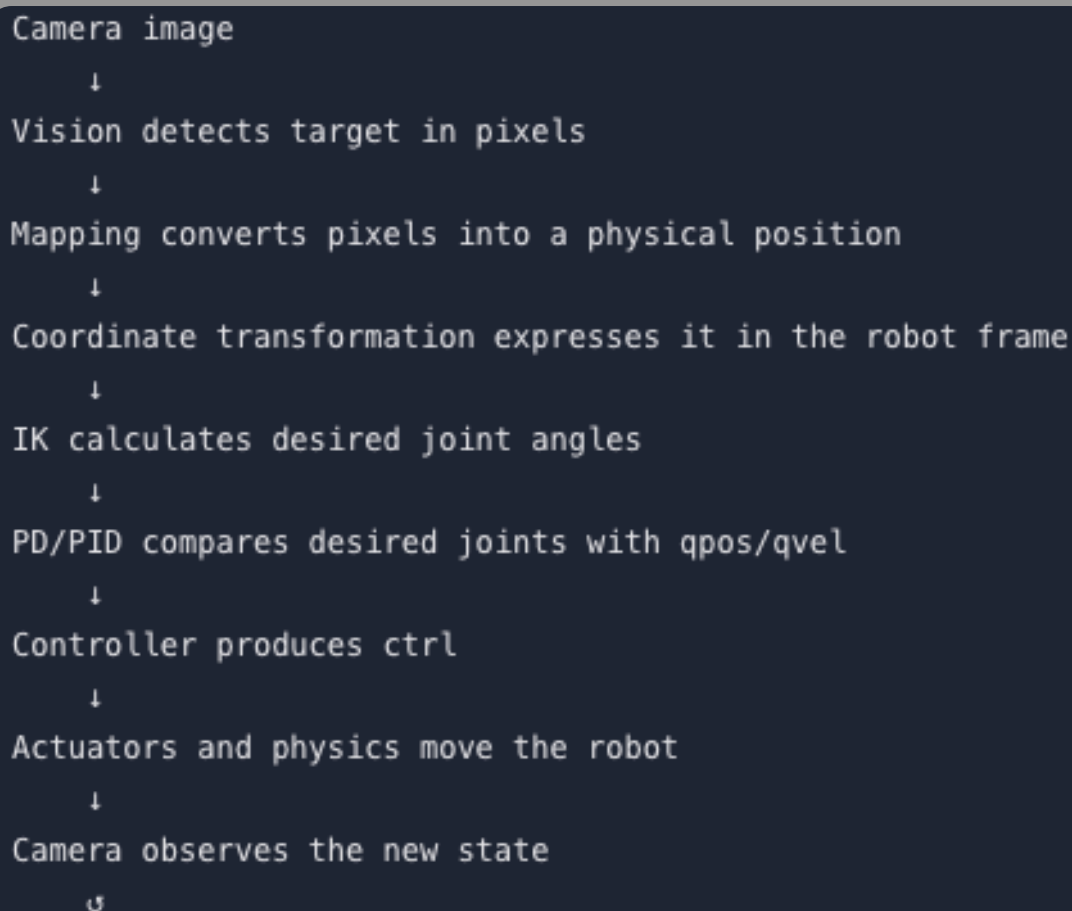
MuJoCo ground truth → IK → controller

for genuine vision-guided robotics
control should use:

Rendered camera image → vision estimate → mapping → IK → controller

$$\text{estimation error} = \text{estimated position} - \text{true position}$$

⇒ Complete Loop (Conclusion)



1. Which stage converts pixels into millimetres?
2. If a valid target is physically outside the arm's reachable area, where should it be rejected?
3. What does PD/PID output?
4. When the camera estimate is wrong by 2 mm, is that control overshoot or perception error?
5. While waiting for the next camera frame, what desired value does the inner controller continue using?

1. mapping : pixels → physical coords.
2. on IK checks reachability
3. ctrl : actuator command

4. perception/estimation error

5. latest desi. joint angles produced by IK